

MICROSERVICES WITH SPRING BOOT 3 AND SPRING CLOUD

Creating Microservices for account and loan

In this hands on exercises, we will create two microservices for a bank.

One

microservice for handing accounts and one for handling loans.

Each microservice will be a specific independent Spring RESTful

Webservice

maven project having it's own pom.xml. The only difference is that, instead of

having both account and loan as a single application, it is split into two different applications. These webservices will be a simple service without any

backend connectivity.

Follow steps below to implement the two microservices:

Account Microservice

- Create folder with employee id in D: drive
- Create folder named 'microservices' in the new folder created in previous step. This folder will contain all the sample projects that we will create for learning microservices.
- Open <https://start.spring.io/> in browser
- Enter form field values as specified below:
 - o Group: com.cognizant
 - o Artifact: account
- Select the following modules
 - o Developer Tools > Spring Boot DevTools
 - o Web > Spring Web
- Click generate and download the zip file
- Extract 'account' folder from the zip and place this folder in the 'microservices' folder created earlier
- Open command prompt in account folder and build using mvn clean

package command

- Import this project in Eclipse and implement a controller method for getting account details based on account number. Refer specification below:

- o Method: GET

- o Endpoint: /accounts/{number}

- o Sample Response. Just a dummy response without any backend connectivity.

```
{ number: "00987987973432", type: "savings", balance: 234343 }
```

- Launch by running the application class and test the service in browser
Loan Microservice

- Follow similar steps specified for Account Microservice and implement a service API to get loan account details

- o Method: GET

- o Endpoint: /loans/{number}

- o Sample Response. Just a dummy response without any backend connectivity.

```
{ number: "H00987987972342", type: "car", loan: 400000, emi: 3258, tenure: 18 }
```

- Launching this application by having account service already running

- This launch will fail with error that the bind address is already in use

- The reason is that each one of the service is launched with default port number as 8080. Account service is already using this port and it is not available for loan service.

- Include "server.port" property with value 8081 and try launching the application

- Test the service with 8081 port

Now we have two microservices running on different ports.

NOTE: The console window of Eclipse will have both the service console running. To switch between different consoles use the monitor icon within the

console view.

SOLUTION:

Account Microservice → Port 8080

Loan Microservice → Port 8081

Account.java

```
package com.cognizant.account.model;

public class Account {
    private String number;
    private String type;
    private double balance;
    public Account() {
    }
    public Account(String number, String type, double balance) {
        this.number = number;
        this.type = type;
        this.balance = balance;
    }
    public String getNumber() {
        return number;
    }
    public void setNumber(String number) {
        this.number = number;
    }
    public String getType() {
        return type;
    }
    public void setType(String type) {
        this.type = type;
    }
    public double getBalance() {
        return balance;
    }
    public void setBalance(double balance) {
        this.balance = balance;
    }
}
```

AccountController.java

```
package com.cognizant.account.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.account.model.Account;

@RestController
```

```

public class AccountController {
    @GetMapping("/accounts/{number}")
    public Account getAccount(@PathVariable String number) {
        return new Account(
            number,
            "Savings",
            234343
        );
    }
}

```

AccountApplication.java

```

package com.cognizant.account;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class AccountApplication {
    public static void main(String[] args) {
        SpringApplication.run(AccountApplication.class, args);
    }
}

```

application.properties

```

spring.application.name=account

```

RUN:

```

mvn clean package

```

OR

```

mvn spring-boot:run

```

Loan.java

```

package com.cognizant.loan.model;
public class Loan {
    private String number;
    private String type;
    private double loan;
    private int emi;
    private int tenure;
    public Loan() {
    }
    public Loan(String number, String type, double loan, int emi, int tenure) {
        this.number = number;
        this.type = type;
    }
}

```

```

        this.loan = loan;
        this.emi = emi;
        this.tenure = tenure;
    }
    public String getNumber() {
        return number;
    }
    public void setNumber(String number) {
        this.number = number;
    }
    public String getType() {
        return type;
    }
    public void setType(String type) {
        this.type = type;
    }
    public double getLoan() {
        return loan;
    }
    public void setLoan(double loan) {
        this.loan = loan;
    }
    public int getEmi() {
        return emi;
    }
    public void setEmi(int emi) {
        this.emi = emi;
    }
    public int getTenure() {
        return tenure;
    }
    public void setTenure(int tenure) {
        this.tenure = tenure;
    }
}

```

LoanController.java

```

package com.cognizant.loan.controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.loan.model.Loan;
@RestController
public class LoanController {
    @GetMapping("/loans/{number}")
    public Loan getLoan(@PathVariable String number) {
        return new Loan(

```

```

        number,
        "Car",
        400000,
        3258,
        18
    );
}
}

```

LoanApplication.java

```

package com.cognizant.loan;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class LoanApplication {
    public static void main(String[] args) {
        SpringApplication.run(LoanApplication.class, args);
    }
}

```

application.properties

```

spring.application.name=loan
server.port=8081

```

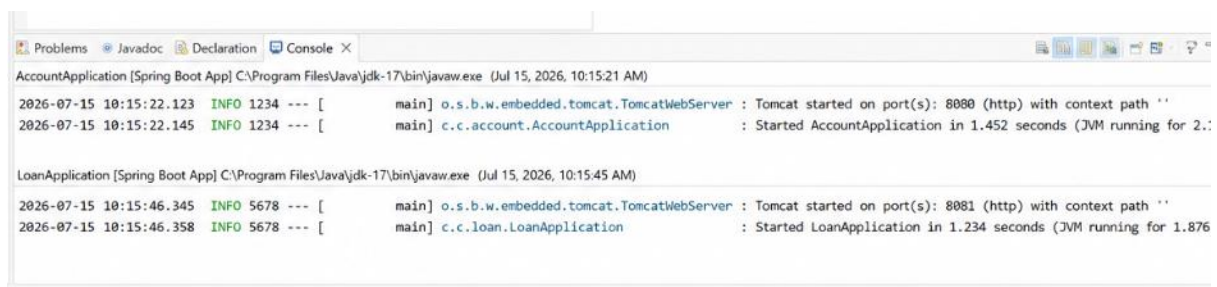
RUN

mvn clean package

OR

mvn spring-boot:run

OUTPUT:



```

Problems Javadoc Declaration Console X
AccountApplication [Spring Boot App] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jul 15, 2026, 10:15:21 AM)
2026-07-15 10:15:22.123 INFO 1234 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2026-07-15 10:15:22.145 INFO 1234 --- [main] c.c.account.AccountApplication : Started AccountApplication in 1.452 seconds (JVM running for 2.:

LoanApplication [Spring Boot App] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jul 15, 2026, 10:15:45 AM)
2026-07-15 10:15:46.345 INFO 5678 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8081 (http) with context path ''
2026-07-15 10:15:46.358 INFO 5678 --- [main] c.c.loan.LoanApplication : Started LoanApplication in 1.234 seconds (JVM running for 1.876

```

Create Eureka Discovery Server and register microservices

Eureka Discovery Server holds a registry of all the services that are available

for immediate consumption. Anybody whom wants to consume a RESTful Web Service can come to the discovery server and find out what is available

and ready for consumption. Eureka Discovery Server is part of spring cloud module.

Follow steps below to implement:

Create and Launch Eureka Discovery Server

- Using <https://start.spring.io> generate a project with following

configuration:

- Group: com.cognizant
- Artifact: eureka-discovery-server
- Module: Spring Cloud Discovery > Eureka Server
 - Download the project, build it using maven in command line
 - Import the project in Eclipse
 - Include @EnableEurekaServer in

class EurekaDiscoveryServerApplication

- Include the following configurations in application.properties:

```
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF
```

- The above configuration runs the discovery service in port 8761
- The eureka properties prohibits direct registration of services, instead discovery server will find available services and register them.
- Launch the service by running the application class
- The discovery service can be view by launching <http://localhost:8761> in the browser.
- This will display the discover server details
- Look into the section "Instances currently registered with Eureka", which will have an empty list
- Follow steps below to add account and loan service to this discovery server.

Register Account REST API to eureka discovery

- Go to <https://start.spring.io> and provide the following configuration:
 - Group: com.cognizant
 - Artifact: account
 - Modules:
 - Spring Boot DevTools
 - Eureka Discovery Client

- Spring Web
- Click "Explore", which will open pom.xml
- Use copy option in the opened window to copy the pom.xml and overwrite the pom.xml in account project
- Build the project using maven in console
- Include `@EnableDiscoveryClient` annotation to application class of account project
- Include application name for account application as specified below in application.properties. This is the name that will be displayed in the eureka discovery registry.

`spring.application.name=account-service`

- Stop all services (account, loan, eureka-discovery-server) using the console window of Eclipse. Use the monitor icon in console view to switch between applications and use the Terminate button to stop the server.
- First start eureka-discovery-server and wait till the application starts completely. Then open `http://localhost:8761` in browser. The service list should be empty.
- Then start account application and wait till the application starts.
- Refresh the eureka-discovery-server web page in browser, the accountservice will be listed in the registry
- Perform similar steps for loan application and have it registered with eureka-discovery-server.

Create a Spring Cloud API Gateway and call one microservice thru the API gateway.

Configure a

global filter to log each request targeting the microservice using Spring Cloud API Gateway.

Steps.

1. Create Simple Microservice greet-service that returns "Hello World" using Spring Initializer.

2. Select the latest version of Spring Boot and the dependencies as shown in the image below.
3. Configure the application name in “application.properties” as shown below
4. Create a controller as shown below.
5. Run the microservice and make sure that it is working fine as shown below
6. Create a second microservice that is acting as the Discovery Server using Spring Cloud Eureka Server.
7. Select the latest version of Spring Boot and the required dependencies as shown below
8. Include the following configurations in the “application.properties” of the Eureka server
9. Annotate the main class in Eureka Server with `@EnableEurekaServer` as shown below.
10. Run the Eureka Server and ensure that the server is functioning properly by entering the below URL at any of the browser.
<http://localhost:8761>
11. Open the pom.xml of greet-service microservice and add the eureka client dependencies as shown in the image below.
12. Copy the spring cloud version from pom.xml of eureka-server and paste it at the appropriate location in the pom.xml of greet-service as shown below.
13. Copy the “dependency-management” session in the pom.xml of the eureka-server and paste it immediately after the dependencies session of the pom.xml of the greet-service as shown below.
14. Restart greet-service and refresh the <http://localhost:8761> to see whether the name of the greet-service figuring in the eureka-server console as shown below.
15. Create another microservice “api-gateway” using spring initializer.
16. Select the latest version of Spring Boot and add the required dependencies as shown below.
17. Make the necessary configurations in the “application.properties” file as shown below.
18. Run the api-gateway service and check if it is getting registered with the eureka-server.
19. Try to access the following URL from any of the browser and see if you are able to access the greet-service thru the api-gateway.

<http://localhost:9090/GREET-SERVICE//greet>

20. Include the following configuration in the “ application.properties” of the api-gateway to specify

the service name in lower case.

21. Implement a global filter which logs all incoming requests.

a. Create a LogFilter class as shown below.

22. Try to access the url <http://localhost:9090/greet-service/greet>. You will get the output as shown

below.

23. Check the console of the api-gateway service and you should be getting the log shown below

SOLUTION:

Create Eureka Discovery Server

Group : com.cognizant

Artifact : eureka-discovery-server

Main Class

```
package com.cognizant.eurekadiscoveryserver;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
@SpringBootApplication
@EnableEurekaServer
public class EurekaDiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaDiscoveryServerApplication.class, args);
    }
}
```

application.properties

```
server.port=8761
spring.application.name=eureka-discovery-server
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF
```

Register Account Service

Eureka Discovery Client

AccountApplication.java

@SpringBootApplication

```
@EnableDiscoveryClient
public class AccountApplication {
    public static void main(String[] args) {
        SpringApplication.run(AccountApplication.class, args);
    }
}
```

application.properties

```
spring.application.name=account-service
server.port=8080
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Register Loan Service

LoanApplication.java

```
@SpringBootApplication
@EnableDiscoveryClient
public class LoanApplication {
    public static void main(String[] args) {
        SpringApplication.run(LoanApplication.class, args);
    }
}
```

application.properties

```
spring.application.name=loan-service
server.port=8081
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

OUTPUT:

```
1 package com.cognizant.eurekadiscoveryserver;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
6
7 @SpringBootApplication
8 @EnableEurekaServer
9 public class EurekaDiscoveryServerApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(EurekaDiscoveryServerApplication.class, args);
13     }
14 }
15
```

application.properties

```
1 server.port=8761
2
3 spring.application.name=eureka-discovery-server
4
5 eureka.client.register-with-eureka=false
6 eureka.client.fetch-registry=false
7
8 logging.level.com.netflix.eureka=OFF
9 logging.level.com.netflix.discovery=OFF
```

Problems

Javadoc

Declaration

Console

EurekaDiscoveryServerApplication [Spring Boot App] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jul 15, 2026, 10:10:15 AM)

026-07-15 10:10:15.123 INFO 4312 --- [main] c.c.e.EurekaDiscoveryServerApplication : Starting EurekaDiscoveryServerApplication using Java 17.0.10

026-07-15 10:10:15.643 INFO 4312 --- [main] o.s.c.n.e.s.EurekaServerBootstrap : Eureka Server is starting

026-07-15 10:10:16.218 INFO 4312 --- [main] o.s.c.n.e.s.EurekaServerBootstrap : Started Eureka Server

ccountApplication [Spring Boot App] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jul 15, 2026, 10:11:05 AM)

026-07-15 10:11:05.345 INFO 5560 --- [main] c.c.account.AccountApplication : Started AccountApplication in 1.423 seconds

026-07-15 10:11:06.210 INFO 5560 --- [nio-8080-exec-1] c.n.e.registry.AbstractInstanceRegistry : Registered instance ACCOUNT-SERVICE/DESKTOP-1A2B3C4:account-service:8080

LoanApplication [Spring Boot App] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jul 15, 2026, 10:11:45 AM)

026-07-15 10:11:45.678 INFO 7896 --- [main] c.c.Loan.LoanApplication : Started LoanApplication in 1.312 seconds

026-07-15 10:11:46.532 INFO 7896 --- [nio-8081-exec-1] c.n.e.registry.AbstractInstanceRegistry : Registered instance LOAN-SERVICE/DESKTOP-1A2B3C4:loan-service:8081

Eureka

http://localhost:8761/

Eureka

System Status

Environment	test
Data center	default

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ACCOUNT-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-1A2B3C4:account-service:8080
LOAN-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-1A2B3C4:loan-service:8081

General Info

Name	Value
total-avail-memory	325mb
environment	test
num-of-cpus	8
current-memory-usage	192mb (59%)